LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

CONSTRUCTOR, DESTRUCTOR, DAN OVERLOADING PADA PHP



Identitas Mahasiswa

Nama	: Diandras Rio Triasmoro
NIM	: 2530521062
Kelas	: 3 TI C

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH SUKABUMI 2026

1. Tujuan Praktikum

- Menjelaskan fungsi constructor, destructor, dan overloading dalam konteks PHP.
- Membuat dan mengakses objek dengan konstruktor berparameter dan nilai default.
- Menganalisis kapan destructor dipanggil dan membedakannya dari operasi Delete pada CRUD.
- Mengimplementasikan __get(), __set(), dan __call() untuk akses dinamis secara terkendali.
- Mengembangkan studi kasus mahasiswa dan menguji hasil melalui browser/localhost.

2. Ringkasan Teori

2.1 Constructor (__construcut)

Constructor adalah metode khusus yang otomatis berjalan ketika sebuah kelas diubah menjadi objek (instansiasi), yaitu saat objek dibuat dengan kata kunci new. Fungsinya mengatur keadaan awal objek, misalnya NIM, nama, dan semester, sehingga objek sudah berisi data yang valid sejak dibuat. Constructor tidak wajib ada, tetapi jika didefinisikan maka argumen saat pembuatan objek harus sesuai dengan parameternya. Parameter dapat diberi nilai default, dan sejak PHP 8 tersedia constructor property promotion yang memungkinkan deklarasi dan pengisian properti dilakukan sekaligus di daftar parameter constructor.

2.2 Destructor (__destruct)

Destructor adalah method khusus yang dipanggil ketika objek dihancurkan, termasuk saat eksekusi skrip berakhir secara normal. Perintah unset($obj) hanya menghapus variabel/referensi, sehingga destructor tidak langsung berjalan jika objek masih memiliki referensi lain (misalnya alias). Urutan pemanggilan destructor di akhir skrip tidak boleh dijadikan ketergantungan logika bisnis. Destructor juga bukan method untuk menghapus data (record) mahasiswa dari Array/Session; fungsi tersebut dilakukan oleh method hapus() pada operasi CRUD.

2.3 Overloading dan Magic Metodhs

Overloading pada PHP berbeda dengan method overloading di Java/C++. PHP tidak memilih salah satu dari beberapa method bernama sama berdasarkan jumlah atau tipe parameter. Di PHP, overloading berarti perilaku dinamis yang dipicu ketika program mengakses property atau method yang tidak ada atau tidak dapat diakses, dengan bantuan magic methods.

Magic methods mempermudah akses dinamis, tetapi dapat menyembunyikan salah ketik dan melemahkan kejelasan struktur class. Atribut inti seperti NIM, nama, dan

semester sebaiknya tetap berupa property dan method eksplisit, sedangkan overloading hanya digunakan untuk atribut tambahan yang memang dirancang dinamis.

3. Source Code dan Hasil Percobaan

3.1 Percobaan Constructor: __construct()

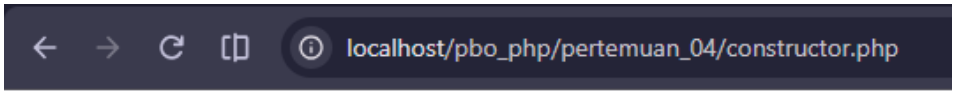
```php
<?php
declare(strict_types=1);

class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1
    ) {}

    public function profil(): string {
        return "$this->nim - $this->nama (Semester $this->semester)";
    }
}

$mhs = new Mahasiswa('231001', 'Alya', 3);
echo $mhs->profil();
```

Output :



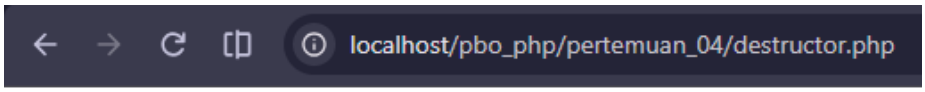
231001 - Alya (Semester 3)

3.2 Percobaan Destructor: __destruct()

```php
<?php
class DemoSiklus {
    public function __construct() { echo "Object dibuat<br>"; }
    public function tampil(): void { echo "Method dipanggil<br>"; }
    public function __destruct() { echo "Destructor berjalan<br>"; }
}

$obj = new DemoSiklus();
$obj->tampil();
unset($obj);
echo "Program dilanjutkan";
```

Output :



Object dibuat
Method dipanggil
Destructor berjalan
Program dilanjutkan

3.3 Percobaan Overloading property

```php
<?php
declare(strict_types=1);

class MahasiswaDinamis {
    private array $atributTambahan = [];

    public function __construct(public string $nim, public string
$nama) {}

    public function __set(string $key, mixed $value): void {
        $diizinkan = ['hobi', 'asalSekolah', 'agama'];
        if (!in_array($key, $diizinkan, true)) {
            throw new InvalidArgumentException('Atribut tidak
diizinkan');
        }
        $this->atributTambahan[$key] = $value;
    }

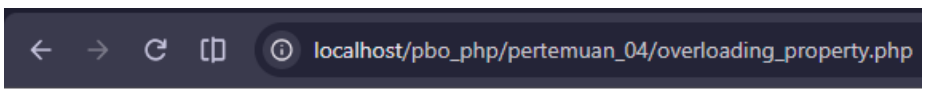
    public function __get(string $key): mixed {
        if (!array_key_exists($key, $this->atributTambahan)) {
            throw new OutOfBoundsException('Atribut tidak ditemukan');
        }
        return $this->atributTambahan[$key];
    }

    public function __isset(string $key): bool {
        return isset($this->atributTambahan[$key]);
    }

    public function __unset(string $key): void {
        unset($this->atributTambahan[$key]);
    }
}

$mhs = new MahasiswaDinamis('231001', 'Alya');
$mhs->hobi = 'Membaca';
echo htmlspecialchars($mhs->hobi, ENT_QUOTES, 'UTF-8');
```

Output :



Membaca

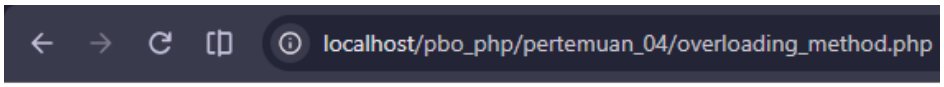
3.4 Percobaan Overloading method dengan __call()

```php
<?php
class LaporanMahasiswa {
    public function __construct(private string $nama) {}

    public function __call(string $method, array $args): mixed {
        if ($method === 'cetakRingkas' && count($args) === 0) {
            return 'Mahasiswa: ' . $this->nama;
        }
        throw new BadMethodCallException('Method tidak tersedia: ' .
$method);
    }
}

$laporan = new LaporanMahasiswa('Alya');
echo $laporan->cetakRingkas();
```

Output :



Mahasiswa: Alya

4. Latihan Mandiri

4.1 Modifikasi constructor Mahasiswa agar memiliki parameter fakultas dengan nilai default “Belum diisi”; buat dua objek dan bandingkan output.

Source code :

```php
<?php
declare(strict_types=1);

// Latihan 1: Constructor Mahasiswa dengan parameter fakultas (default
"Belum diisi")
class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1,
        public string $fakultas = "Belum diisi"
    ) {}

    public function profil(): string {
        return "$this->nim - $this->nama (Semester $this->semester) |
Fakultas: $this->fakultas";
    }
}

// Objek 1: fakultas TIDAK diisi -> memakai nilai default
```

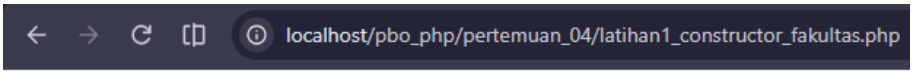
```php
$mhs1 = new Mahasiswa('231001', 'Alya', 3);

// Objek 2: fakultas diisi
$mhs2 = new Mahasiswa('231002', 'Budi', 5, 'Sains dan Teknologi');

echo "Objek 1 (tanpa fakultas): " . htmlspecialchars($mhs1->profil())
. "<br>";
echo "Objek 2 (dengan fakultas): " . htmlspecialchars($mhs2->profil())
. "<br>";
```

Output :

Perbandingan parameter default membuat argumen fakultas bersifat opsional saat objek dibuat.



Objek 1 (tanpa fakultas): 231001 - Alya (Semester 3) | Fakultas: Belum diisi
Objek 2 (dengan fakultas): 231002 - Budi (Semester 5) | Fakultas: Sains dan Teknologi

4.2 Buat class Buku dengan judul, penulis, dan tahun melalui constructor, serta method tampilInfo(); buat tiga objek.

Source code :

```php
<?php
declare(strict_types=1);

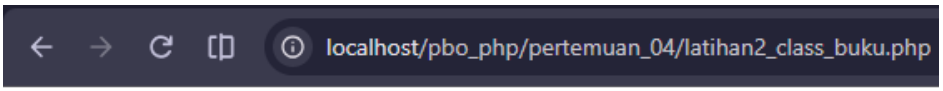
// Latihan 2: Class Buku dengan constructor dan method tampilInfo()
class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
    ) {}

    public function tampilInfo(): string {
        return "Judul: $this->judul | Penulis: $this->penulis | Tahun:
$this->tahun";
    }
}

$buku1 = new Buku('Pemrograman Berorientasi Objek dengan PHP', 'Andi
Wijaya', 2021);
$buku2 = new Buku('Dasar-Dasar Basis Data', 'Siti Rahmawati', 2019);
$buku3 = new Buku('Algoritma dan Struktur Data', 'Budi Santoso',
2020);

echo "1. " . htmlspecialchars($buku1->tampilInfo()) . "<br>";
echo "2. " . htmlspecialchars($buku2->tampilInfo()) . "<br>";
echo "3. " . htmlspecialchars($buku3->tampilInfo()) . "<br>";
```

Output :



1. Judul: Pemrograman Berorientasi Objek dengan PHP | Penulis: Andi Wijaya | Tahun: 2021
2. Judul: Dasar-Dasar Basis Data | Penulis: Siti Rahmawati | Tahun: 2019
3. Judul: Algoritma dan Struktur Data | Penulis: Budi Santoso | Tahun: 2020

4.3 Tambahkan destructor demonstrasi pada class Buku; bandingkan efek unset ketika masih ada alias object.

Source code :

```php
<?php
declare(strict_types=1);

// Latihan 3: Destructor pada class Buku + pengaruh alias terhadap
unset()
class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
    ) {
        echo "[Constructor] Buku '$this->judul' dibuat<br>";
    }

    public function tampilInfo(): string {
        return "Judul: $this->judul | Penulis: $this->penulis | Tahun:
$this->tahun";
    }

    public function __destruct() {
        echo "[Destructor] Buku '$this->judul' dihancurkan<br>";
    }
}


// Skenario A: unset TANPA alias
echo "<b>Skenario A: unset tanpa alias</b><br>";
$bukuA = new Buku('Buku A', 'Penulis A', 2020);
unset($bukuA);
echo "Setelah unset(\$bukuA) -> destructor langsung berjalan (sudah
tampil di atas)<br><br>";

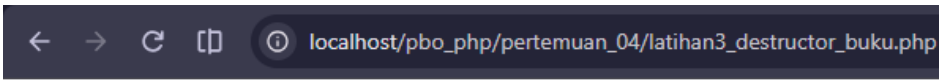
// Skenario B: unset DENGAN alias
echo "<b>Skenario B: unset dengan alias</b><br>";
```

```php
$bukuB = new Buku('Buku B', 'Penulis B', 2021);
$alias = $bukuB;                    // objek kini punya 2 referensi
unset($bukuB);                      // hanya menghapus 1 referensi
echo "Setelah unset(\$bukuB) -> destructor BELUM berjalan, objek masih
bisa diakses lewat alias:<br>";
echo htmlspecialchars($alias->tampilInfo()) . "<br>";
unset($alias);                      // referensi terakhir dihapus
echo "Setelah unset(\$alias) -> destructor berjalan (sudah tampil di
atas)<br><br>";

// Skenario C: tanpa unset -> destructor jalan di akhir skrip
echo "<b>Skenario C: tanpa unset</b><br>";
$bukuC = new Buku('Buku C', 'Penulis C', 2022);
echo "Akhir skrip... destructor Buku C akan berjalan setelah
ini.<br>";
```

Output :

- Skenario A (tanpa alias): destructor langsung berjalan saat unset() dipanggil.
- Skenario B (dengan alias): setelah unset($bukuB), destructor belum berjalan dan objek masih bisa diakses lewat $alias. Destructor baru berjalan setelah unset($alias).
- Skenario C (tanpa unset): destructor berjalan di akhir skrip.



Skenario A: unset tanpa alias
[Constructor] Buku 'Buku A' dibuat
[Destructor] Buku 'Buku A' dihancurkan
Setelah unset($bukuA) -> destructor langsung berjalan (sudah tampil di atas)

Skenario B: unset dengan alias
[Constructor] Buku 'Buku B' dibuat
Setelah unset($bukuB) -> destructor BELUM berjalan, objek masih bisa diakses lewat alias:
Judul: Buku B | Penulis: Penulis B | Tahun: 2021
[Destructor] Buku 'Buku B' dihancurkan
Setelah unset($alias) -> destructor berjalan (sudah tampil di atas)

Skenario C: tanpa unset
[Constructor] Buku 'Buku C' dibuat
Akhir skrip... destructor Buku C akan berjalan setelah ini.
[Destructor] Buku 'Buku C' dihancurkan

4.4 Tambahkan atribut dinamis “hobi” pada class latihan dan uji __set(), __get(), __isset(), dan __unset().

Source code :

```php
<?php
declare(strict_types=1);

// Latihan 4: Atribut dinamis "hobi" dengan __set, __get, __isset,
__unset
class MahasiswaDinamis {
    private array $atributTambahan = [];
```

```php
    public function __construct(public string $nim, public string
$nama) {}

    public function __set(string $key, mixed $value): void {
        $diizinkan = ['hobi'];
        if (!in_array($key, $diizinkan, true)) {
            throw new InvalidArgumentException("Atribut '$key' tidak
diizinkan");
        }
        $this->atributTambahan[$key] = $value;
        echo "[__set] atribut '$key' diisi<br>";
    }

    public function __get(string $key): mixed {
        echo "[__get] membaca atribut '$key'<br>";
        if (!array_key_exists($key, $this->atributTambahan)) {
            throw new OutOfBoundsException("Atribut '$key' tidak
ditemukan");
        }
        return $this->atributTambahan[$key];
    }

    public function __isset(string $key): bool {
        echo "[__isset] memeriksa atribut '$key'<br>";
        return isset($this->atributTambahan[$key]);
    }

    public function __unset(string $key): void {
        echo "[__unset] menghapus atribut '$key'<br>";
        unset($this->atributTambahan[$key]);
    }
}

$mhs = new MahasiswaDinamis('231001', 'Alya');

// 1. Uji __set
echo "<b>1. Uji __set()</b><br>";
$mhs->hobi = 'Membaca';

// 2. Uji __get
echo "<b>2. Uji __get()</b><br>";
echo "Hobi: " . htmlspecialchars($mhs->hobi, ENT_QUOTES, 'UTF-8') .
"<br>";

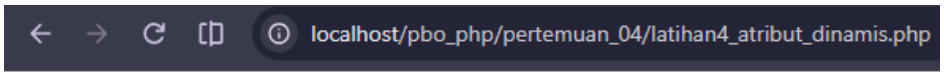
// 3. Uji __isset
echo "<b>3. Uji __isset()</b><br>";
echo "isset(hobi)? " . (isset($mhs->hobi) ? 'Ya' : 'Tidak') . "<br>";
```

```php
// 4. Uji __unset
echo "<b>4. Uji __unset()</b><br>";
unset($mhs->hobi);
echo "isset(hobi) setelah unset? " . (isset($mhs->hobi) ? 'Ya' :
'Tidak') . "<br>";

// 5. Uji atribut yang tidak diizinkan
echo "<b>5. Uji atribut tidak diizinkan</b><br>";
try {
    $mhs->alamat = 'Sukabumi';
} catch (InvalidArgumentException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br>";
}

// 6. Uji membaca atribut yang sudah dihapus
echo "<b>6. Uji membaca hobi setelah di-unset</b><br>";
try {
    echo $mhs->hobi;
} catch (OutOfBoundsException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output :



1. Uji __set()
[__set] atribut 'hobi' diisi
2. Uji __get()
[__get] membaca atribut 'hobi'
Hobi: Membaca
3. Uji __isset()
[__isset] memeriksa atribut 'hobi'
isset(hobi)? Ya
4. Uji __unset()
[__unset] menghapus atribut 'hobi'
[__isset] memeriksa atribut 'hobi'
isset(hobi) setelah unset? Tidak
5. Uji atribut tidak diizinkan
Exception: Atribut 'alamat' tidak diizinkan
6. Uji membaca hobi setelah di-unset
[__get] membaca atribut 'hobi'
Exception: Atribut 'hobi' tidak ditemukan

4.5 Buat __call() untuk menangani satu nama method yang diizinkan; uji pemanggilan method lain dan catat exception.

Source code :

```php
<?php
declare(strict_types=1);

// Latihan 5: __call() yang hanya mengizinkan satu nama method
class LaporanMahasiswa {
    public function __construct(private string $nama) {}

    public function __call(string $method, array $args): mixed {
```

```
        // Hanya method "cetakRingkas" tanpa argumen yang diizinkan
        if ($method === 'cetakRingkas' && count($args) === 0) {
            return 'Mahasiswa: ' . $this->nama;
        }
        throw new BadMethodCallException('Method tidak tersedia: ' .
$method);
    }
}

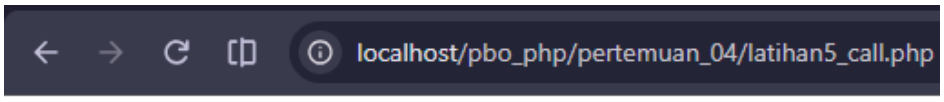
$laporan = new LaporanMahasiswa('Alya');

// 1. Method yang diizinkan
echo "<b>1. Method yang diizinkan (cetakRingkas)</b><br>";
echo htmlspecialchars($laporan->cetakRingkas()) . "<br><br>";

// 2. Method lain yang tidak diizinkan
echo "<b>2. Method lain (cetakLengkap)</b><br>";
try {
    echo $laporan->cetakLengkap();
} catch (BadMethodCallException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) .
"<br><br>";
}

// 3. Method diizinkan tapi dengan argumen
echo "<b>3. cetakRingkas dengan argumen</b><br>";
try {
    echo $laporan->cetakRingkas('tambahan');
} catch (BadMethodCallException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output :



1. Method yang diizinkan (cetakRingkas)
Mahasiswa: Alya

2. Method lain (cetakLengkap)
Exception: Method tidak tersedia: cetakLengkap

3. cetakRingkas dengan argumen
Exception: Method tidak tersedia: cetakRingkas

5. Tabel Hasil Pengujian

Kasus	Gejala yang diharapkan	Analisis	Hasil Pengujian
Jumlah parameter constructor salah	ArgumentCountError	Kontrak konstruktor tidak dipenuhi	Muncul ArgumentCountError (Too few arguments to function Mahasiswa::__construct()) ketika objek dibuat tanpa argumen wajib, misalnya new Mahasiswa('231001').
Tipe parameter salah (strict)	TypeError	Tipe argumen tidak cocok	Muncul TypeError karena argumen semester harus bertipe int, tetapi diberi string non-numerik (misalnya "tiga").
Akses atribut dinamis tidak diizinkan	InvalidArgumentException	Validazione di __set()	Muncul InvalidArgumentException dengan pesan “Atribut tidak diizinkan” saat menulis atribut selain hobi, asalSekolah, dan agama.
Method tidak dikenali __call()	BadMethodCallException	Jangan menelan kesalahan secara diam-diam	Muncul BadMethodCallException dengan pesan “Method tidak tersedia: ” diikuti nama method yang dipanggil.
unset satu referensi, masih ada alias	Destructor belum dipanggil	Object masih direferensikan	Setelah unset($obj), pesan destructor belum muncul. Pesan baru muncul setelah unset($alias) atau saat skrip berakhir.

6. Analisis Urutan Constructor-Metohd-Destructor

Berdasarkan Percobaan Destructor dan Latihan 4.3, siklus hidup objek berjalan dengan urutan berikut:

1. Constructor dijalankan otomatis saat objek dibuat dengan new, sehingga pesan "Object dibuat" muncul lebih dahulu.
2. Method biasa dijalankan ketika dipanggil secara eksplisit melalui operator ->, misalnya $obj->tampil().
3. Destructor dijalankan ketika tidak ada lagi referensi yang menunjuk ke objek, atau ketika skrip selesai dieksekusi.

Pada Percobaan Destructor, unset($obj) tanpa alias membuat jumlah referensi objek menjadi nol, sehingga destructor langsung berjalan dan teks "Destructor berjalan" muncul sebelum "Program dilanjutkan". Jika sebelumnya dibuat $alias = $obj, unset($obj) hanya mengurangi satu referensi sehingga objek tetap hidup. Destructor baru berjalan setelah unset($alias), yaitu ketika referensi terakhir dihapus. Jika tidak ada unset sama sekali, destructor berjalan di akhir skrip.

Hasil ini menunjukkan bahwa destructor berkaitan dengan siklus hidup objek di memori, bukan dengan penghapusan data. Karena itu destructor berbeda dengan method hapus() pada CRUD yang menghapus record mahasiswa dari Array/Session, dan destructor tidak boleh dipakai sebagai mekanisme penyimpanan data Session.

7. Analisis Pemicu Magic Methods

Magic methods pada overloading hanya terpicu ketika property atau method yang diakses tidak tersedia atau tidak dapat diakses dari luar class. Hasil Percobaan Overloading property, Percobaan Overloading method, serta Latihan 4.4 dan 4.5 dirangkum sebagai berikut :

Magic method	Pemicu	Pengamatan pada percobaan
__set()	Menulis nilai ke property yang tidak tersedia/tidak terakses	$mhs->hobi = "Membaca" memanggil __set(); atribut di luar daftar yang diizinkan memunculkan InvalidArgumentException.
__get()	Membaca property yang tidak tersedia/tidak terakses	echo $mhs->hobi memanggil __get(); atribut yang tidak ada memunculkan OutOfBoundsException.

__isset()	Memanggil isset() atau empty() pada property dinamis	isset($mhs->hobi) mengembalikan true setelah diisi dan false setelah di-unset.
__unset()	Memanggil unset() pada property dinamis	unset($mhs->hobi) menghapus nilai dari array atributTambahan.
__call()	Memanggil method instance yang tidak tersedia/tidak terakses	cetakRingkas() ditangani oleh __call(); method lain dan pemanggilan dengan argumen memunculkan BadMethodCallException.

Magic methods bermanfaat untuk atribut tambahan yang bersifat dinamis dan terkendali, misalnya hobi atau asal sekolah, karena class tidak perlu mendeklarasikan setiap property. Risikonya, salah ketik nama property atau method tidak langsung terlihat sebagai error, dan struktur class menjadi kurang jelas. Oleh karena itu validasi pada __set() dan exception pada __get() dan __call() dibutuhkan agar akses tetap terkendali.

8. Evaluasi dan Refleksi

8.1 Apa perbedaan constructor dengan method biasa?

Constructor (__construct) adalah metode khusus yang dieksekusi secara otomatis ketika sebuah objek pertama kali dibuat menggunakan keyword new. Sebaliknya, method biasa tidak berjalan secara otomatis dan harus dipanggil secara manual menggunakan operator ->.

8.2 Mengapa nilai awal objek sebaiknya valid sejak dibuat?

Agar objek siap digunakan dengan keadaan (state) yang konsisten dan terhindar dari error. Dengan menetapkan nilai awal yang valid melalui constructor (seperti NIM dan nama), kita memastikan bahwa kontrak pembuatan objek telah terpenuhi sebelum program memanipulasi data objek tersebut.

8.3 Apakah unset($obj) selalu langsung menjalankan destructor? Jelaskan.

Tidak selalu. Perintah unset($obj) hanya menghapus variabel atau referensi tertentu terhadap objek. Jika objek tersebut masih memiliki referensi atau alias lain yang aktif di memori, destructor (__destruct) tidak akan berjalan hingga semua referensi tersebut dihapus atau eksekusi program benar-benar berakhir.

8.4 Mengapa destructor tidak sama dengan method hapus() dalam CRUD?

Destructor hanya menangani pembersihan objek dari memori (RAM) komputer pada saat eksekusi skrip PHP berakhir. Method hapus() pada CRUD berfungsi untuk

menghapus data atau record secara permanen dari media penyimpanan persisten (seperti Session, Array global, atau Database). Oleh karena itu, destructor tidak boleh dijadikan mekanisme untuk menghapus data bisnis.

8.5 Apa perbedaan overloading PHP dengan method overloading di Java/C++?

Dalam bahasa Java atau C++, overloading berarti mendefinisikan beberapa method dengan nama yang sama di dalam satu kelas, tetapi dibedakan berdasarkan jumlah atau tipe parameternya. Di PHP, overloading merujuk pada fitur yang dipicu melalui magic methods seperti __call, __get, __set ketika program mencoba mengakses atau memanggil properti maupun method yang tidak tersedia atau berstatus private/protected.

8.6 Kapan penggunaan __get/__set bermanfaat, dan apa risikonya?

Bermanfaat: Ketika kita perlu menyimpan atau mengakses atribut tambahan yang sifatnya dinamis atau opsional misalnya data ekstra seperti 'hobi' atau 'agama' yang tidak selalu ada.
Risiko: Dapat melemahkan kejelasan struktur kelas dan berpotensi menyembunyikan bug. Misalnya, jika programmer salah mengetik nama properti, program tidak akan memunculkan pesan error, melainkan diam-diam membuat atau mengakses properti dinamis yang salah.

9. Kesimpulan

Constructor adalah metode khusus yang otomatis berjalan ketika sebuah kelas diubah menjadi objek, dan fungsinya mengatur keadaan awal objek sehingga datanya valid sejak dibuat. Parameter default dan constructor property promotion membuat inisialisasi lebih ringkas. Destructor berjalan ketika objek dihancurkan, yaitu saat tidak ada lagi referensi ke objek atau saat skrip berakhir. Karena itu unset() tidak selalu langsung memanggil destructor jika objek masih punya alias, dan destructor tidak sama dengan operasi hapus pada CRUD karena tugasnya mengatur siklus hidup objek, bukan menghapus data.

Overloading pada PHP adalah penanganan dinamis lewat magic methods (__get, __set, __isset, __unset, dan __call) ketika property atau method yang diakses tidak tersedia, sehingga berbeda dengan method overloading pada Java/C++. Magic methods bermanfaat untuk atribut tambahan yang memang dirancang dinamis, tetapi berisiko menyembunyikan salah ketik dan membuat struktur class kurang jelas. Oleh karena itu atribut inti tetap sebaiknya dibuat eksplisit, sedangkan akses dinamis perlu disertai validasi dan exception agar tetap terkendali.